

Salient Object Detection: An End-to-End Deep Learning Project

Arbison Krasniqi - 8th May, 2026

1. Introduction

Salient Object detection is a computer vision task whose main focus is to identify regions/areas of an image that stand out or draw human visual attention. Unlike similar object detection models, whose goal is to identify all objects in a scene, SOD outputs a single binary mask that highlights only the most visually dominant region. This makes a segmentation problem, where each pixel is assigned a value classifying whether it belongs to the salient region or not.

SOD is used in a wide range of applications, some of which include image compression, automatic cropping, thumbnail generation, visual tracking, and content-aware editing.

This work reflects the design and implementation of a complete SOD system from scratch, using foundational machine learning and deep learning concepts. The project covers the full pipeline: a custom data loader with augmentation, a convolutional encoder-decoder model implemented in PyTorch, a training loop with checkpointing, an evaluation module that computes IoU, precision, recall, and F1 score, and an interactive demo built with Gradio. A baseline architecture was implemented first, then improved through several modifications including batch normalization, data augmentation, and a switch to a U-Net architecture with skip connections. The same models were evaluated on two public datasets, ECSSD and MSRA10K, in order to study how the system responds to differences in dataset size. In total, six experiments were run to document the progression from the initial baseline to the final improved version.

2. Background

Encoder-Decoder Architecture

Most modern image segmentation models follow an encoder-decoder structure. The encoder takes the input image and starts by “zooming out” or reducing the image size while increasing the number of feature channels. This creates a compact image representation that doesn’t have fine details but is full of features.

The decoder does the opposite.

It takes the encoding result and “zooms back in,” by increasing its resolution until it reaches the original image size and produces a segmentation mask. The turning point between the encoding and decoding phase is called the bottleneck. All of the information about the input image has to go through this bottleneck before the decoder rebuilds it back into a full sized output.

The main weakness of this setup is that spatial detail gets lost on the way down and is hard to recover on the way up since you'd be working with a much smaller resolution image. The decoder is essentially trying to redraw edges and boundaries from a heavily compressed summary, and the result is often a blocky/blurry mask that doesn't line up cleanly with the actual object.

U-Net and Skip Connections

U-Net was made for medical image segmentation, and it solves the bottleneck problem by using skip connections. At each step of the decoder, the output of the matching encoder layer is added back to the decoder's current feature map before the next step. This way the decoder doesn't only have the compressed bottleneck information to work with, it also gets the details from the preceding encoder layers that came before. U-Net networks produce sharper masks with cleaner edges than a regular encoder-decoder model does.

Loss Functions

For binary segmentation tasks like SOD, two loss functions are usually combined to train the model. The first is Binary Cross-Entropy (BCE), which compares the predicted probability of each pixel against the ground truth, treating every pixel as it's own classification problem. BCE works well when the foreground and background are roughly balanced, but on it's own it can underweight the salient region when it only takes up a small portion of the image.

To make up for this, an IoU based term is usually added on top. IoU measures how well the predicted mask overlaps with the ground truth mask as a whole region. The combined loss used in this project is $BCE + 0.5 \times (1 - \text{IoU})$.

Evaluation Metrics

Four metrics are used to evaluate the model on the test set. IoU is the same measure used in the loss, and reports the overlap between the predicted and ground truth masks as a region. Precision reports the fraction of predicted salient pixels that are actually salient. Recall reports the fraction of true salient pixels that the model successfully predicts. F1 score is the harmonic mean of precision and recall, and gives a single number that balances the two.

Reporting all four together gives a more complete picture than any single metric on it's own, since a model can have high precision but low recall (only predicting the obvious pixels and missing the rest), or the opposite (predicting too aggressively and including non salient pixels in the result).

3. Dataset

Two public datasets were used in this project, ECSSD and MSRA10K. Both are standard benchmarks for salient object detection and come with binary masks where each pixel is either part of the salient region (255) or background (0).

ECSSD (Extended Complex Scene Saliency Dataset) contains 1000 images each with their saliency masks. It was used for the initial experiments, since it's small enough to iterate on quickly while still being challenging enough to test whether the model is actually learning something.

MSRA10K is a larger dataset with 10000 images, also with their respective saliency masks. They were added later in the project, to see how the same model would perform when given ten times more images to train on. The filenames in MSRA10K are different compared to ECSSD, so a small Python script was written to rename them into the same format used in the ECSSD dataset and split them into separate folders. This way the existing data loader could be used with the new dataset without changing the code.

All images and masks are resized to 128×128 during loading and the pixel values are normalized to the range 0–1. The dataset is split into 70% training, 15% validation, and 15% testing, using a fixed random seed so the split is reproducible across runs.

Data augmentation is applied only to the training set, and includes a random horizontal flip and a small random rotation. The validation and test sets are kept clean, so the metrics reflect the model's actual performance and not the effect of augmentation.

4. Methodology

4.1 Baseline Architecture

The baseline model is a simple encoder-decoder model. The encoder has four blocks, each contains one Conv2D layer followed by a ReLU activation. After each block, a 2×2 MaxPool layer is applied to make the resolution smaller. The number of feature channels grows from 32 in the first block to 256 in the last block.

The decoder has four ConvTranspose2D layers with ReLU activations. Each layer doubles size while halving the number of channels, until the output reaches the original 128×128 resolution. A final 1×1 convolution followed by a Sigmoid activation produces the single channel saliency mask, with each pixel value between 0 and 1 representing the model's confidence that the pixel is salient.

This model does not use batch normalization or skip connections.

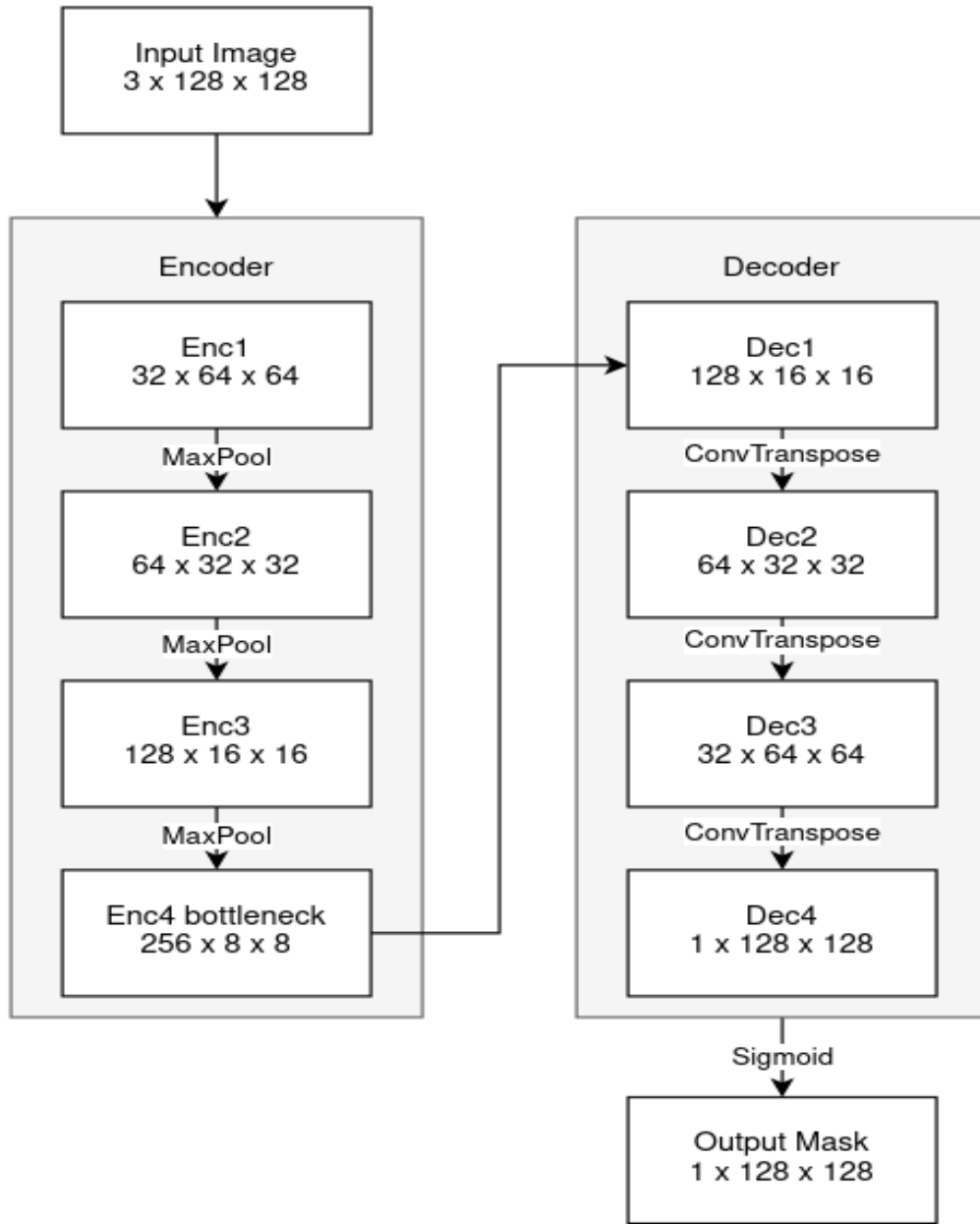


Figure 1. Baseline SOD Model

4.2 U-Net Architecture

The improved architecture is a U-Net implementation, defined in `UNET_SOD_MODEL.py`. The overall encoder-decoder shape is the same, but with two important changes.

First, the channel sizes are larger. The encoder grows from 64 to 512 channels (compared to 32 to 256 in the baseline), giving the model more capacity to learn rich features at each level.

Second, skip connections are added between the encoder and decoder. At each step of the decoder, the output of the matching encoder layer is concatenated to the decoder's current feature map along the channel dimension before the next convolution. This way the decoder doesn't only have the bottleneck

information to work with, it also has the spatial details from the earlier encoder layers (see Section 2 for the full explanation).

Each convolutional block in the U-Net also includes a Batch Normalization layer between the convolution and the ReLU activation, which helps stabilize training and was found to improve results in the experiments described in Section 5.

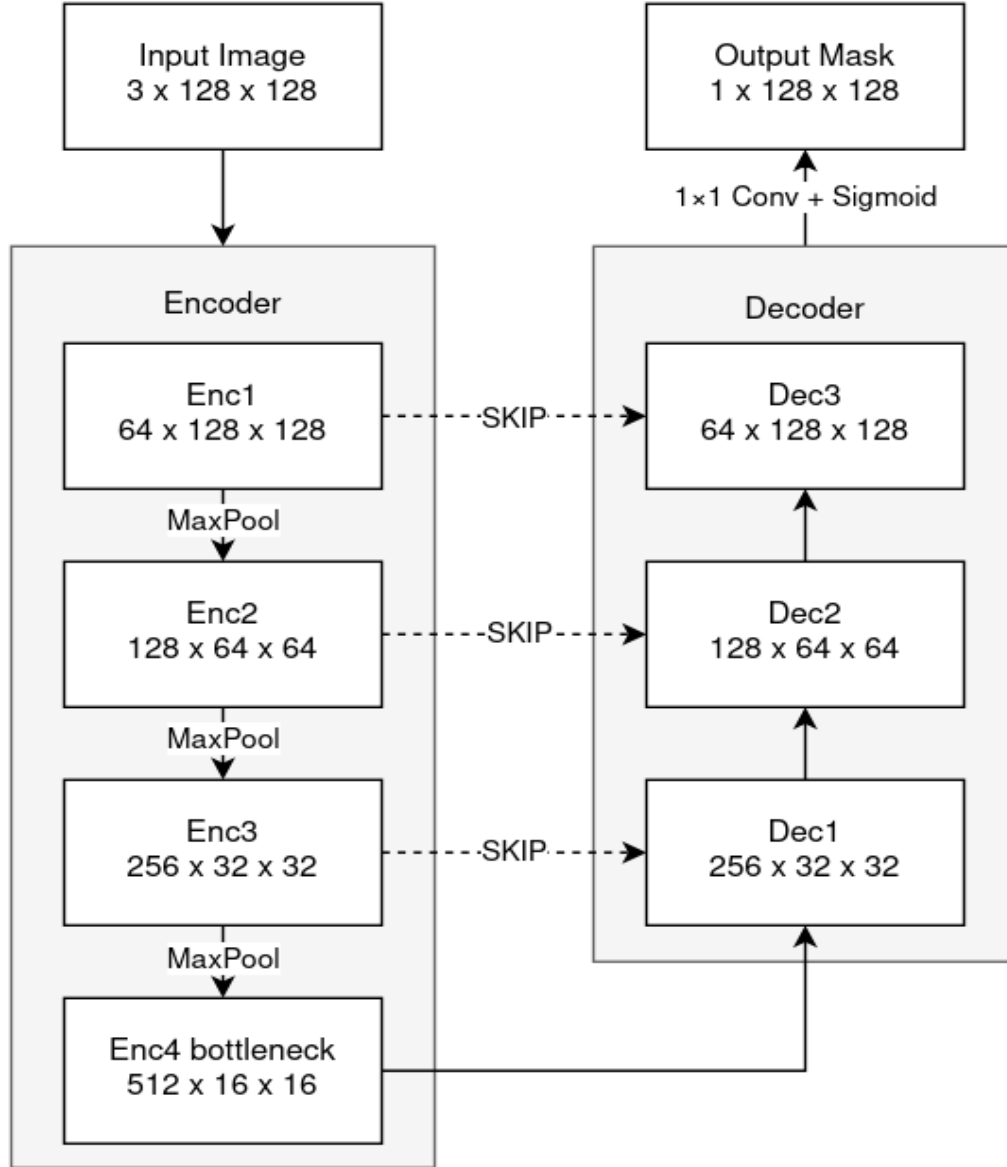


Figure 2. U-Net SOD Model. Solid arrows show the main encoder-decoder path, dotted arrows show skip connections between matching encoder and decoder layers.

4.3 Training Setup

Both models were trained using the same setup, so the comparisons between experiments are fair. The optimizer is Adam with a learning rate of $1e-3$, and the batch size is 32. Each model was trained for 20 epochs.

The loss function is the combined $\text{BCE} + 0.5 \times (1 - \text{IoU})$ described in Section 2. Validation is run after each training epoch, and the model with the lowest validation loss across all epochs is saved as the final checkpoint in checkpoints/best_model.pth.

The training loop also prints per-epoch training and validation loss to the console, so the progression can be monitored during the run and the early stopping point can be identified after the fact.

5. Experiments and Results

To measure how each change affected the model's performance, six experiments were run. All six used the same training setup: Adam optimizer with a learning rate of $1e-3$, batch size of 32, 20 epochs, the combined $\text{BCE} + 0.5 \times (1 - \text{IoU})$ loss, and a 70/15/15 train/validation/test split with a fixed random seed. The only things that change between runs are the variables being tested (image size, batch normalization, data augmentation, dataset, and architecture), so any difference in results can be attributed to those changes.

The results across all six runs are summarized in the table below. Each row reports the metrics on the test set after the model with the lowest validation loss has been selected.

Run	Model	Dataset	BN	Aug	Img Size	IoU	Precision	Recall	F1
1	Baseline	ECSSD	no	no	128x128	0.3296	0.5894	0.4315	0.4938
2	Baseline	ECSSD	no	no	224x224	0.3012	0.5387	0.4092	0.4609
3	Baseline	ECSSD	yes	no	128x128	0.4028	0.6236	0.5322	0.5737
4	Baseline	ECSSD	yes	yes	128x128	0.4309	0.5981	0.6094	0.6013
5	Baseline	MSRA10K	yes	yes	128x128	0.6110	0.7141	0.8087	0.7578
6	U-Net	MSRA10K	yes	yes	128x128	0.6780	0.7945	0.8221	0.8075

Run 1: Baseline

The first run uses the baseline model at 128×128 with no batch normalization and no augmentation, on the ECSSD dataset. The result is an F1 of 0.4938, which is an okay starting point. The model is clearly learning something, but the masks it produces are still rough and the validation loss flattens out fairly early in training.

Run 2: Larger Image Size

For the second run the only change was the image size, going from 128×128 to 224×224 . The thinking was that more pixels means more detail and more information for the model to work with. The result was actually slightly worse (F1 0.4609 vs 0.4938). The most likely reason is that the model capacity didn't scale with the input. The same architecture working on 3 times the pixels per image trains slower per epoch and has more spatial information to compress through the same bottleneck, so 20 epochs aren't enough to reach convergence. After this run, all later experiments went back to 128×128 .

Run 3: Adding Batch Normalization

The third run added batch normalization to every convolutional block. BN works by normalizing the outputs of each layer before they get passed forward, which keeps the activations from drifting too far during training and lets the optimizer take more confident steps. The improvement was significant, with F1 jumping from 0.4938 to 0.5737. This was the first run where the validation loss kept improving throughout training instead of flattening early.

Run 4: Adding Data Augmentation

For the fourth run, augmentation was added to the training set in the form of a random horizontal flip and a small random rotation. Augmentation works by showing the model slightly different versions of the same image each epoch, which helps it generalize and not just memorize the training set. The improvement here was smaller, with F1 going from 0.5737 to 0.6013, but the gap between training loss and validation loss got noticeably smaller, which is a sign that the model was overfitting less.

Run 5: Switching to MSRA10K

The fifth run kept the same model and the same training setup, but used MSRA10K instead of ECSSD. MSRA10K has 10000 images compared to ECSSD's 1000, so the model now has 10 times more data to learn from. The result was the biggest single jump of the whole project, with F1 going from 0.6013 to 0.7578. What's interesting about this run is that nothing about the model itself changed. The improvement came entirely from giving the same architecture more data to work with. This is a useful reminder that for many problems, more data matters more than tweaking the model.

Run 6: U-Net Architecture

The final run replaced the baseline architecture with the U-Net described in Section 4. With skip connections, the decoder now has access to the spatial details from the earlier encoder layers and not just the compressed bottleneck. The result was a further improvement, with F1 reaching 0.8075. The U-Net's masks have visibly cleaner edges than the baseline's, which makes sense given that the whole point of skip connections is to preserve spatial detail through the network.

Visualization

Figure 3 shows the predictions of the best model (Run 6) on a random batch of test images. From left to right: the input image, the ground truth mask, the predicted mask, and an overlay of the prediction on the input. The model successfully identifies the salient object in most cases, with cleaner edges than what the baseline architecture produced.

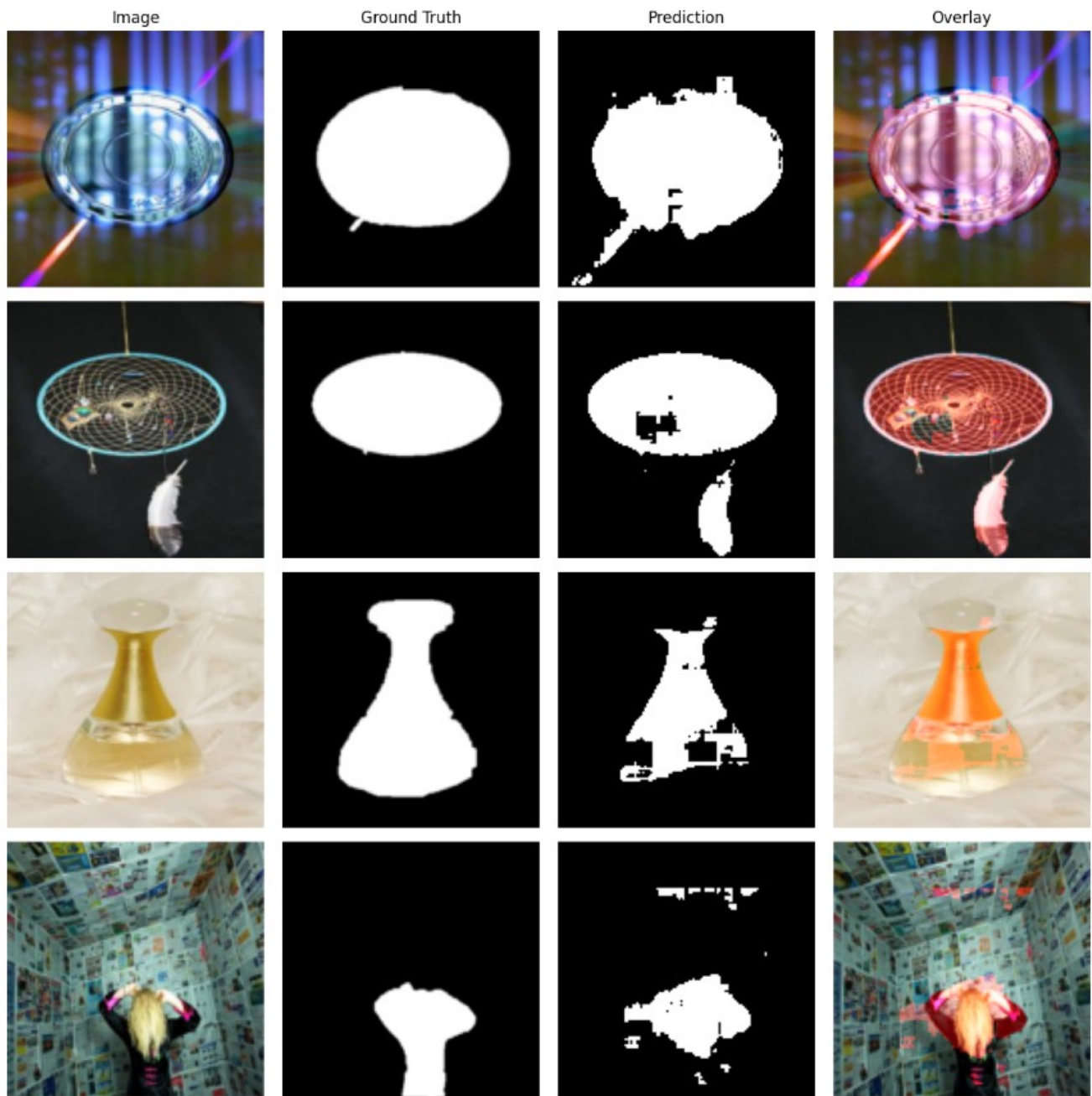


Figure 3: Sample predictions of the U-Net model on the MSRA10K test set. Columns from left to right: input image, ground truth mask, predicted mask, overlay.

Summary

The progression across the six runs shows a clear pattern. The baseline model at 128×128 starts at F1 0.4938, and each change tested adds an improvement on top of the previous one. Batch normalization helped the most among the architectural changes, augmentation gave a smaller but real boost, switching to a larger dataset was the single biggest improvement, and finally moving to a U-Net architecture pushed the result to its highest point at F1 0.8075. The most important takeaway is that data quantity ended up mattering more than any single model tweak, which is consistent with what is generally observed in deep learning.

6. Demo

To make the model easy to interact with, a small web app was built using Gradio. The app loads the trained U-Net checkpoint at startup, then accepts any image the user uploads and returns three things: the predicted saliency mask, an overlay of the prediction on top of the original image, and the inference time in milliseconds. The whole app is defined in a single file (app.py).

The flow inside the app is straightforward. When the user uploads an image, it gets resized to 128×128 and normalized to the 0-1 range, the same way the training data was preprocessed. It then gets a batch dimension added, gets moved to the GPU if one is available, and gets passed through the model. The output is binarized at a threshold of 0.5, resized back to the original image size using nearest-neighbor interpolation (so the mask edges stay sharp instead of getting blurred), and used to build the overlay by adding a red tint on the salient pixels. The inference time is measured around the model's forward pass only, not around the preprocessing, so the number reflects the actual model's speed.

Loading the model once at startup instead of on every request makes the app feel responsive, and on a GPU the inference itself usually takes around 2-3 milliseconds per image.

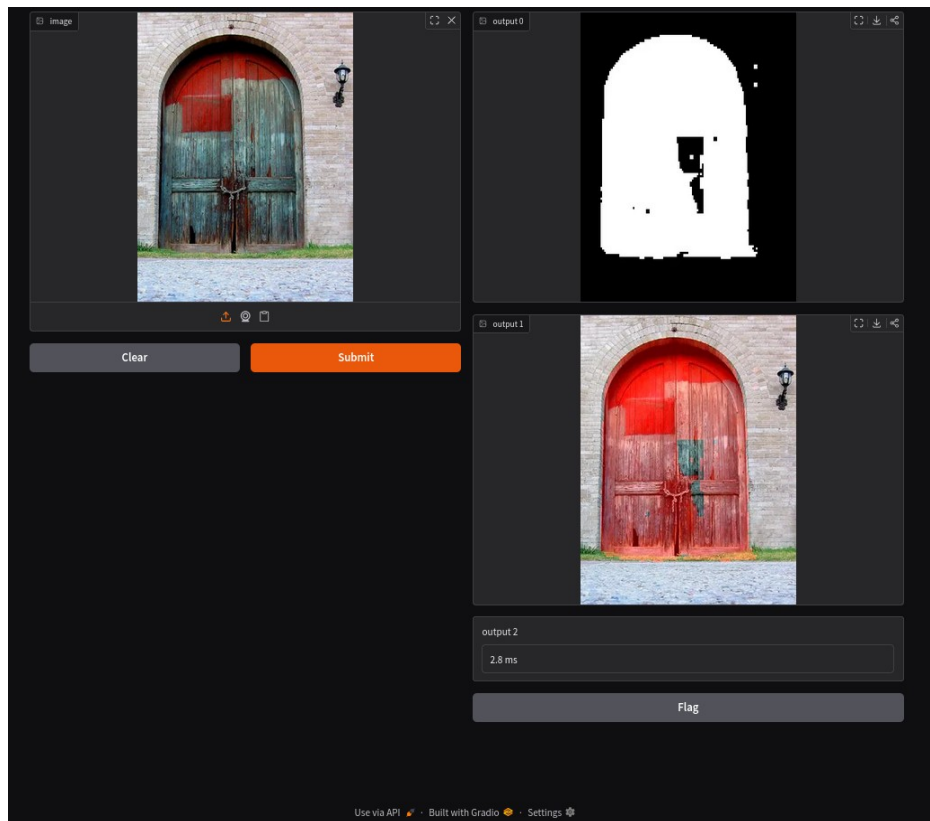


Figure 4: The Gradio demo running on a sample image. The user uploads an image and immediately sees the predicted mask, an overlay highlighting the salient region, and the model's inference time.

7. Discussion and Conclusion

The six experiments show a clear pattern. Each change helped, but not equally. Batch normalization gave the biggest jump among the architectural changes, augmentation gave a smaller but real improvement, and the U-Net with skip connections produced cleaner masks at the end. The single biggest improvement of the whole project, though, didn't come from any model change at all. It came from switching from ECSSD to MSRA10K and giving the same model 10 times more data, which is a useful reminder that for a lot of problems, more data matters more than tweaking the model. The Run 2 result, where increasing the image size actually made the model worse, was useful in a different way, showing that not every change that sounds like an improvement actually is one.

The most valuable thing learned from building this from scratch is on the implementation side. Setting up the convolutional layers manually and seeing how the same Conv + BatchNorm + ReLU pattern can be packed into a small reusable block and chained together to build something larger, gave a much better understanding of how a model is actually put together than any tutorial would. The same modular thinking made it possible to swap out the dataset, the augmentation, or the architecture without rewriting the rest of the code. The final U-Net model on MSRA10K reaches an F1 of 0.8075 on the test set, which is a solid result for a model built entirely from scratch without pretrained weights, and more importantly, the process of building each component manually made the deep learning pipeline feel concrete instead of abstract.

Sources

- **MSRA10K dataset** — Cheng et al. (2015), "Global Contrast based Salient Region Detection"
Project page: <https://mmcheng.net/msra10k/>
- **PyTorch documentation** <https://pytorch.org/docs/>
- **Gradio documentation** <https://www.gradio.app/docs>
- **U-Net** — overview of the architecture and why it improves on standard encoder-decoder designs <https://en.wikipedia.org/wiki/U-Net>
- **ECSSD dataset** — Extended Complex Scene Saliency Dataset, 1000 images with pixel-accurate saliency masks <https://www.cse.cuhk.edu.hk/leojia/projects/hsaliency/dataset.html>